Zest

Simplified Scientific Computing on Distributed Infrastructure

Execute scientific tools on HPC/cloud infrastructure
with a single command

White Paper — Version 1.0

Status: Early Development

Akash Network, Open Source CLI

Target Audience: CNES, Aix-Marseille University, CMA CGM, Scientific Communities

Contents

1 Executive Summary

Zest is an open-source CLI tool designed to simplify the execution of scientific workloads on distributed infrastructure (Akash Network, traditional HPC, cloud).

The goal: *eliminate the complexity* associated with containerization, environment configuration, and file transfers.

1.1 The Problem

Researchers and engineers running intensive scientific pipelines face:

- Manual configuration of Docker/VMs
- Complex file transfer management
- Learning multiple tools (AWS CLI, Slurm, etc.)
- Infrastructure administration that consumes valuable time
- Vendor lock-in with single cloud providers

1.2 The Solution

A single CLI interface, **tool-centric**, that:

- Hides infrastructure complexity
- Automates transfer, provisioning, and cleanup
- Supports pre-packaged tools (bioinformatics, geospatial) and custom scripts
- Works on decentralized (Akash) or traditional infrastructure
- Remains fully open-source and extensible

1.3 Benefits for Researchers and Enterprises

1.3.1 For Researchers

- **Accelerated Research**: Less time on infrastructure, more on science. Move from idea to results in minutes, not weeks.
- **Reproducible Science**: Containerized tools ensure consistent execution environments across institutions.
- **Global Collaboration**: Shared tool registry enables seamless collaboration between international research teams.
- **Access to HPC**: Democratizes access to high-performance computing for researchers worldwide.

1.3.2 For Enterprises

- **Cost Optimization**: Pay only for compute resources used, with no upfront infrastructure investment.
- **Operational Efficiency**: Eliminates DevOps overhead for scientific workloads.
- **Scalability**: Easily scale compute resources up or down based on project needs.
- **Innovation Acceleration**: Focus on core business while leveraging external HPC resources.

1.3.3 For the Global Scientific Community

- **Democratization of Computing**: Researchers in developing countries can access world-class infrastructure without massive capital investment.
- **Decentralized Infrastructure**: Akash Network distributes computing globally, reducing geopolitical dependencies and local costs.
- **Reproducible Science**: Open-source code + public CLI = total transparency. Everyone can verify, reproduce, and improve.
- **Global Community**: Creates links between resource-rich institutions and labs with limited resources. Enables frictionless sharing of scientific pipelines.
- **Sustainable Impact**: Free and accessible tools = more inclusive science, more robust results (global peer review).

Free CLI + Managed Service The CLI is open-source and free. An optional web service will offer job management, history, multi-user support, and institutional integration.

2 Technical Architecture

2.1 Overview

HPC Flow operates on a simple three-layer architecture:

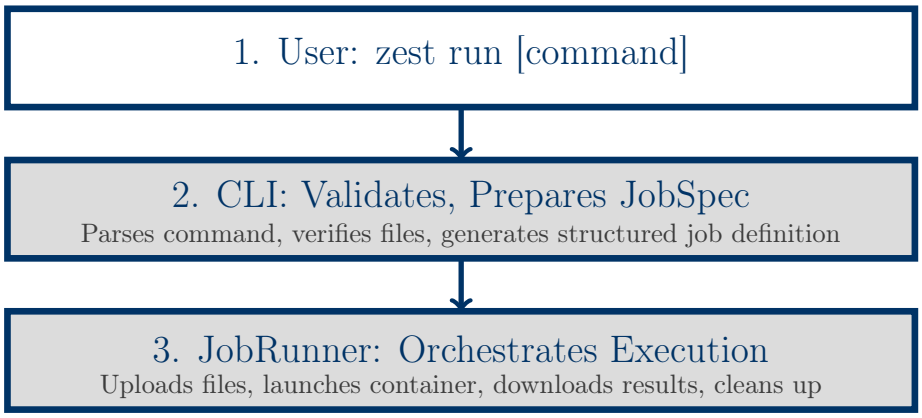

Figure 1: Simplified Execution Flow

2.2 Key Components

2.2.1 1. CLI Layer

The Command-Line Interface (CLI) is the primary user interaction point. It:

- Parses user commands (e.g., `zest run vsearch -input data.fasta`)
- Validates input files and parameters
- Constructs a **JobSpec** (structured job definition)
- Selects the appropriate backend (Akash, AWS, Slurm, etc.)
- Submits the job to the JobRunner

2.2.2 2. JobSpec Definition

A structured YAML/JSON document defining:

```
tool: vsearch
version: "2.21.1"
input_files:
- metagenome_1M_reads.fasta
output_files:
- centroids.fasta
parameters:
id: 0.97
threads: 16
runtime: "docker://quay.io/biocontainers/vsearch:2.21.1"
backend: akash
```

2.2.3 3. JobRunner Abstraction

The core abstraction that handles backend-specific execution:

```
class JobRunner(ABC):
@abstractmethod
def submit(self, job_spec: JobSpec) -> str:
"""Submit job to backend and return job_id"""
pass

@abstractmethod
def monitor(self, job_id: str) -> str:
"""Monitor job execution and return status"""
pass

@abstractmethod
def download(self, job_id: str, local_path: str) -> None
   :
"""Download output files (job_id) -> local_files"""
pass

@abstractmethod
def cleanup(self, job_id: str) -> None:
"""Clean up backend resources"""
pass
```

2.2.4 4. Backend Implementations

Phase 1 (MVP): AkashJobRunner

- Uploads input files to Akash
- Creates an Akash deployment with the tool image
- Monitors execution via Akash logs
- Downloads results
- Destroys the deployment

Phase 2+: AWS, GCP, OVH, Slurm Support

- **AWSJobRunner**: Uses Lambda or EC2
- **GCPJobRunner**: Uses Cloud Run or Compute Engine
- **SlurmJobRunner**: Integrates with traditional HPC

2.2.5 5. Custom Script Execution

For scientists with specific pipelines:

```
zest run-script my_analysis.py \
--runtime python:3.11 \
--input data.csv \
--output result.html \
--cloud akash
```

The script and its dependencies (`requirements.txt`) are uploaded and executed in a base runtime.

3 Execution Workflow

3.1 Flow of a `hpc run` Command

1. **Parsing**: CLI interprets the command and flags
2. **Validation**: Checks file existence, flag validity, available resources
3. **JobSpec Construction**: Creates a structured job description
4. **Backend Selection**: Chooses the JobRunner (Akash, AWS, etc.)
5. **Resource Provisioning**: Allocates resources on the backend
6. **File Upload**: Transfers input files to remote
7. **Job Execution**: Launches the container with arguments
8. **Monitoring**: Polls status, captures logs
9. **Result Download**: Retrieves output files locally
10. **Cleanup**: Destroys the deployment

Total Duration: A few seconds (provisioning) + compute time + a few seconds (download).

4 Target Use Cases

4.1 Bioinformatics

4.1.1 Sequence Analysis (CNES, Universities)

Evolutionary biology researchers need clustering of thousands of DNA sequences.

```
zest run vsearch \
--input metagenome_1M_reads.fasta \
--id 0.97 \
--threads 16 \
--output centroids.fasta \
--cloud akash
```

Advantage: One command. No infrastructure administration.

4.1.2 BLAST / Alignment (Biotech, University)

```
zest run blast \
--query proteins.fasta \
--db nr \
--output results.tsv \
--cloud akash
```

4.2 Geospatial

4.2.1 DEM Raster Processing (CNES, CMA CGM Logistics)

Calculating slopes, aspects, vegetation indices on satellite imagery.

```
zest run geoprocess \
--input dem_region.tif \
--operation slope \
--output slope_map.tif \
--cloud akash
```

CMA CGM Use Case: Logistics optimization via terrain relief analysis for route planning.

4.3 Custom Scripts

4.3.1 Proprietary Data Analysis

Researcher with a specific pipeline (combination of in-house tools + standard libraries).

```
zest run-script proprietary_analysis.py \
--runtime python:3.11 \
--input confidential_data.csv \
--output report.html \
--cloud akash
```

The `requirements.txt` file is automatically downloaded.

5 Akash Network: Primary Backend and Multi-Cloud Strategy

5.1 What is Akash Network?

Akash Network is a decentralized cloud computing marketplace built on blockchain technology. Unlike traditional cloud providers (AWS, Google Cloud, Azure), Akash operates as an open marketplace where computing resources are traded peer-to-peer.

Key characteristics:

- **Decentralized**: No single entity controls the infrastructure
- **Open Market**: Providers compete to offer resources, ensuring transparent pricing
- **Blockchain-Based**: Smart contracts automate resource allocation and payment
- **Container-First**: Native support for Docker containers

5.2 Why Akash for Scientific Computing?

Criteria	Akash	AWS	GCP
Decentralized	✓	×	×
60-80% Cheaper	✓	Baseline	≈ AWS
No Vendor Lock-in	✓	×	×
Open Market (peer-to-peer)	✓	×	×
Easy Container Integration	✓	✓	✓

Table 1: Akash vs. Centralized Cloud Comparison

1. Decentralization and Sovereignty Akash is a decentralized blockchain. No single actor controls the infrastructure. Ideal for:

- **CNES**: No geopolitical dependencies for spatial data
- **CMA CGM**: Sensitive logistics data not centralized
- **Universities**: Flexible data governance

2. Cost Efficiency Prices **60-80% lower** than AWS/GCP due to the peer-to-peer model. Providers on Akash optimize their costs.

3. No Vendor Lock-in Multi-backend architecture allows switching to another cloud without changing user code. Sustainable competitive advantage.

4. Open Market Multiple providers bid for jobs. Price transparency. No captive pricing.

5.3 Akash Network Current Limitations and Roadmap

Akash Network is evolving with several major improvements planned:

- **Enhanced GPU Support**: Advanced support for NVIDIA H100, A100, and H200 GPUs with performance optimization for AI/ML and intensive computing
- **Improved SLA Guarantees**: Implementation of formal, automated SLA guarantees for better deployment reliability

- **Decentralized Storage Solution**: Deployment of an S3-compatible storage solution with native integration into the Akash ecosystem
- **Provider Ecosystem**: Significant expansion of providers with enhanced quality of service mechanisms and improved reputation systems
- **Monitoring and Observability Tools**: Unified dashboard and advanced tools for deployment monitoring and provider evaluation
- **Cost Optimization**: Continuous reduction in deployment costs with dynamic pricing mechanisms

5.3.1 Akash HomeNode - MVP: Towards Decentralized Participatory Science

Akash Network has announced the development of **Akash HomeNode**, an MVP solution aimed at democratizing access to decentralized computing resources. This initiative represents a major opportunity for **participatory science** by enabling:

- **Citizen Distributed Computing**: Transforming home computers into compute nodes contributing to scientific projects (medical research, astronomy, climatology, etc.)
- **Accessible Infrastructure**: Reducing barriers to entry for researchers and citizens wishing to participate in distributed computing initiatives
- **Practical Applications**:
  - Biomedical data analysis
  - Climate modeling
  - Rare disease research
  - Astronomical data processing

This approach aligns with Akash Network's philosophy of creating a truly **democratic and decentralized** cloud infrastructure where individuals can contribute to scientific advancement while earning compensation for shared resources.

5.4 Multi-Cloud Strategy and Backend Selection

Long-term Vision: Zest abstracts the infrastructure. The user chooses the backend based on need.

5.4.1 Backend Selection: Per Command

The user specifies the backend directly in the command:

```
# Use Akash (default, cost-effective)
zest run vsearch --input data.fasta --cloud akash

# Use AWS for critical job requiring 99.99\% SLA
zest run blast --input proteins.fasta --cloud aws --
    instance-type c5.4xlarge

# Use traditional HPC for high-performance GPU
zest run deeplearning --input train.tar --cloud slurm --
    partition gpu
```

5.4.2 Backend Selection: Per User Profile

Phase 2: Persistent configuration in `~/.hpc/config.yaml`:

```
# Default profile
default_cloud: akash

# Overrides by job type
cloud_rules:
- match:
tool: blast
input_size: ">10GB"
cloud: aws

- match:
tool: deeplearning
cloud: aws
gpu: true
instance_type: p3.8xlarge

- match:
runtime: python
memory_required: ">16Gi"
cloud: slurm
partition: gpu
```

Advantages:

- **Flexibility**: Scientist chooses the best cloud for each use case
- **Cost Optimization**: Akash by default for standard jobs, OVH or other centralized actors for critical workloads
- **No Relearning**: Consistent CLI across all backends

5.5 Target Infrastructures

5.5.1 Akash Network

- **Ideal for**: Cost-sensitive workloads, decentralized computing, containerized tools
- **Use Cases**: Bioinformatics, geospatial analysis, custom scripts
- **Advantages**: 60-80% cost reduction, no vendor lock-in, global compute distribution

5.5.2 AWS

- **Ideal for**: Critical workloads requiring high SLA (99.99%), GPU-intensive tasks, large-scale data processing
- **Use Cases**: High-throughput sequencing, deep learning, large-scale simulations
- **Advantages**: Mature ecosystem, extensive service catalog, enterprise-grade SLAs

5.5.3 Traditional HPC (Slurm)

- **Ideal for**: Massively parallel MPI workloads, institutions with internal infrastructure, on-premise data sensitivity
- **Use Cases**: Climate modeling, fluid dynamics, genomics pipelines
- **Advantages**: High-performance computing, existing infrastructure integration, data locality

5.5.4 SecNumCloud

- **Ideal for**: French entities requiring sovereign cloud compliance
- **Use Cases**: Government agencies, biotech/pharma with sensitive patient data, 100% France-hosted infrastructure
- **Advantages**: French regulatory compliance, data sovereignty, high security standards

6 Business Model

6.1 Open-Source CLI (Free)

- MIT license source code
- Local installation: `pip install hpc-flow`
- Unlimited usage (infrastructure costs borne by user)
- Community contributions welcome

6.2 Managed Service (Paid / SaaS)

Phase 2-3: Optional web platform for:

- **Multi-User Management**: Teams, projects, quotas
- **History and Monitoring**: Jobs, logs, duration, costs
- **Web Dashboard**: Job submission, monitoring, results
- **API Integration**: Programmatic access
- **Technical Support**: Guaranteed SLA

Estimated Pricing (Phase 3) Freemium model: free up to X jobs/month, then usage-based pricing or institutional subscription.

7 Status and Roadmap

7.1 Phase 1 — MVP / POC (Ongoing)

Goal: Functional CLI + Akash backend + 2-3 demo tools.

✓ Generic JobRunner architecture
✓ AkashJobRunner implementation
✓ CLI parsing and JobSpec
✓ Tool registry (vsearch, geoprocess)
✓ File transfer (upload/download)
✓ Script execution (Python)
- POC demonstration (bioinformatics + geospatial)

Timeline: 2-3 months

7.2 Phase 2 — Hardening and Multi-Cloud

Goal: Production-ready CLI, AWS + Slurm support, profile configuration.

- Additional tools (BLAST, SAMtools, complete GDAL)
- Script runtimes: Bash, R, Julia
- Local job history (`zest log`, `zest history`)
- AWSJobRunner implementation
- SlurmJobRunner implementation
- Config file support (~/.hpc/config.yaml)
- Cloud selection logic
- Robust error handling

Timeline: 3-4 months after Phase 1

7.3 Phase 3 — Managed Layer and Multi-Cloud

Goal: Optional SaaS, multi-cloud support (GCP, SecNumCloud, Azure).

- Web app / dashboard
- User and project management
- GCPJobRunner implementation
- SecNumCloud integration (partnership)
- Azure support
- REST API
- Pricing engine (cost estimation)
- Advanced resource prediction

Timeline: 4-6 months after Phase 2

8 Competitive Advantages

Criteria	Zest	Classic Cloud	Traditional HPC
No Docker/VM Required	✓	×	×
Tool-Centric Interface	✓	×	×
Decentralized / Multi-Cloud	✓	×	×
Automated File Transfer	✓	Manual	Manual
Automatic Cleanup	✓	Manual	Manual
Open-Source and Extensible	✓	×	Partial
Pay-as-You-Go	✓	✓	×

8.1 Comparison with Alternatives

8.1.1 vs. Classic Cloud Providers (AWS, GCP, Azure)

- **Complexity**: Classic clouds require Docker/VM management, Zest abstracts this
- **Cost**: 60-80% savings with Akash backend
- **Flexibility**: Multi-cloud architecture prevents vendor lock-in
- **Accessibility**: Designed for scientists, not DevOps engineers

8.1.2 vs. Traditional HPC

- **Access**: HPC requires institutional access, Zest democratizes access
- **Setup**: HPC requires Slurm/PBS expertise, Zest uses simple CLI
- **Integration**: HPC is rigid, Zest supports any containerized tool
- **Cost**: HPC requires upfront investment, Zest is pay-as-you-go

8.1.3 vs. Other Scientific Computing Platforms

- **Simplicity**: Other platforms require complex setup, Zest uses one command
- **Extensibility**: Open registry vs. closed ecosystems
- **Community**: Open-source vs. proprietary solutions

8.2 Target Organizations

8.2.1 French National Centre for Space Studies (CNES)

- **Challenge**: Processing massive satellite data volumes, high-performance computing needs
- **Zest Contribution**: Simplified access to HPC resources, cost optimization, containerized tools for spatial data processing
- **Use Cases**: Image processing, climate modeling, orbital mechanics simulations
- **Benefits**: Reduced infrastructure management, faster time-to-results, collaboration facilitation

8.2.2 Aix-Marseille University

- **Challenge**: Supporting diverse research fields (biology, physics, computer science) with varying compute needs
- **Zest Contribution**: Unified interface for different scientific domains, cost-effective compute access
- **Use Cases**: Genomics research, climate data analysis, AI/ML experiments
- **Benefits**: Democratized HPC access, reduced IT overhead, cross-disciplinary collaboration

8.2.3 CMA CGM (Maritime Logistics)

- **Challenge**: Optimizing routes, weather predictions, on-demand compute without over-capacity
- **Zest Contribution**: Easy integration into data pipelines, elastic scaling, proportional costs, cloud choice (Akash or internal integration)
- **Use Cases**: Terrain relief analysis (marine routes), weather predictions, route optimization
- **Benefits**: Cost flexibility, no over-provisioning, rapid deployment

8.2.4 Akash Network

- **Challenge**: Increasing adoption of decentralized computing, providing high-level tools
- **Zest Contribution**: Killer app for scientists, abstracting Akash complexity, bootstrapping scientific community
- **Synergies**: Co-marketing, Akash API integration, joint case studies, accelerated provider adoption

9 Risks and Mitigation

Risk	Description	Mitigation
Akash Network Immaturity	Price variability, compute availability	Multi-cloud fallback
Slow Adoption	Researchers accustomed to existing tools	Public POC + demos, institutional partnerships
Registry Fragmentation	Too many tools, poor maintenance	Clear governance, guidelines, CI/CD, code review
Akash Scaling Limits	Bottleneck for large file transfers	S3 URLs, streaming, multi-cloud fallback
User Support Demand	Growing technical support requests	Managed SaaS Phase 3, excellent docs, community
Backend Compatibility	Divergent syntax AWS vs GCP	Uniform JobRunner abstraction

10 Conclusion

HPC Flow addresses a real and universal need: simplifying the execution of scientific workloads without infrastructure expertise, while preserving **freedom of choice** in infrastructure.

10.1 Key Strengths

1. **Simplicity**: One command for everything. No Docker, no VMs, no hassle.
2. **Flexibility**: Open-source, extensible, multi-cloud by design.
3. **Decentralization**: Akash Network as primary backend, others possible.
4. **Cost Savings**: 60-80% reduction in compute costs vs. centralized cloud (Akash Phase 1).
5. **Community**: Contributive registry, MIT license, welcome contributions.
6. **Sustainable Model**: Free CLI + optional SaaS.
7. **Multi-Cloud**: Never locked in. Avoid vendor lock-in.

10.2 Next Steps

- Phase 1: Functional Akash POC (end Q2 2024)
- Partnerships: CNES, Aix-Marseille University, Akash Network
- Publication + presentations at scientific conferences (JRES, ADP)
- Recruitment of open-source contributors
- Phase 2: Multi-cloud support (AWS, Slurm) end Q4 2024

HPC Flow: Making scientific computing accessible to all, with infrastructure choice and optimized costs.

A Technical Glossary

Term	Definition
HPC	High Performance Computing
Akash Network	Peer-to-peer decentralized computing resource marketplace
CLI	Command-Line Interface
JobSpec	Structured job specification (tool, args, files, resources)
JobRunner	Abstraction for submitting jobs to a backend (Akash, AWS, Slurm)
Registry	Catalog of tool definitions
POC	Proof of Concept
YAML	Human-readable data serialization format
API	Application Programming Interface
SLA	Service Level Agreement
P2P	Peer-to-Peer
Vendor Lock-in	Dependency on a single provider without exit strategy
SecNumCloud	French sovereign cloud security framework

B References

- GitHub Repository: https://github.com/hdavid-13/hpc-flow
- Akash Network: https://akash.network
- Akash Roadmap: https://github.com/akash-network/roadmap
- Biocontainers: https://biocontainers.pro
- SecNumCloud: https://www.ssi.gouv.fr/entreprise/qualifications/